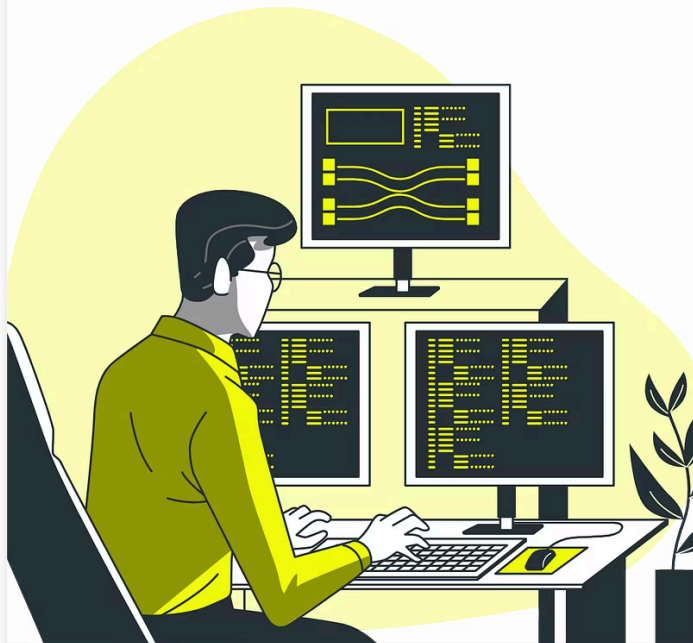




Java

Clase 11 | Desarrollo de la API REST con Spring Boot

¡Les damos la bienvenida! 🚀

🎥 Vamos a comenzar a grabar la clase.

Índice

Clase 11

Clase 12

Clase 11 | Desarrollo de la API REST con Spring Boot

- Concepto de API REST y métodos HTTP.
- Creación de controladores (@RestController).
- Manejo de rutas y endpoints básicos.
- Envío y recepción de datos en formato JSON.
- Implementación de operaciones CRUD en memoria.

Clase 12 | Integración con Base de Datos MySQL

- Introducción a bases de datos relacionales y MySQL.
- Instalación y configuración de MySQL y Workbench.
- Creación de la base de datos para el proyecto.
- Conexión de Spring Boot con MySQL usando Spring Data JPA.
- Mapeo de entidades y relaciones básicas.

Objetivos de la Clase



Comprender API REST

Entender el concepto, principios y uso de métodos HTTP para distintas operaciones.



Crear Controladores

Utilizar `@RestController` en Spring Boot para exponer endpoints.



Configurar Rutas

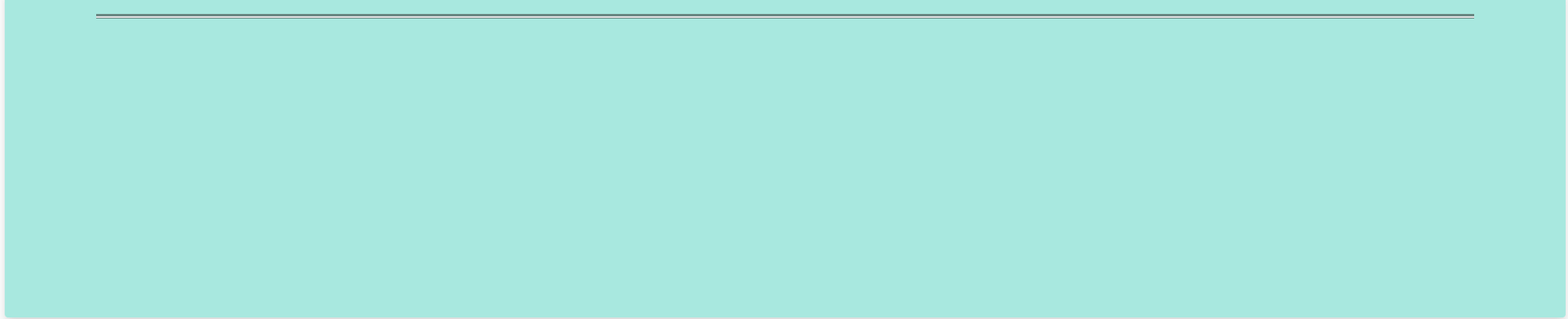
Establecer rutas para operaciones CRUD básicas de productos.



Trabajar con JSON

Aprovechar la serialización automática de Spring Boot para manejar datos JSON.

API REST



API REST

Una API REST (Representational State Transfer) es un estilo arquitectónico para el diseño de servicios web que se basa en la transferencia de representaciones de estado. Se caracteriza por su simplicidad, escalabilidad y uso de estándares web como HTTP.

En una API REST, los recursos, o datos que se manejan, se identifican mediante URLs, y las operaciones que se realizan sobre ellos se mapean a métodos HTTP específicos (GET, POST, PUT, DELETE), permitiendo una interacción clara y eficiente entre un cliente y un servidor.



Métodos HTTP en API REST

GET

El método GET se utiliza para obtener o recuperar un recurso específico. Se envía una solicitud al servidor con la URL del recurso, y el servidor responde con la representación del recurso en un formato determinado, generalmente JSON o XML.

POST

El método POST se emplea para crear un nuevo recurso. Se envía una solicitud al servidor con la URL donde se creará el recurso, junto con los datos necesarios para su creación. El servidor crea el recurso y responde con una confirmación, o en algunos casos con la representación del recurso recién creado.

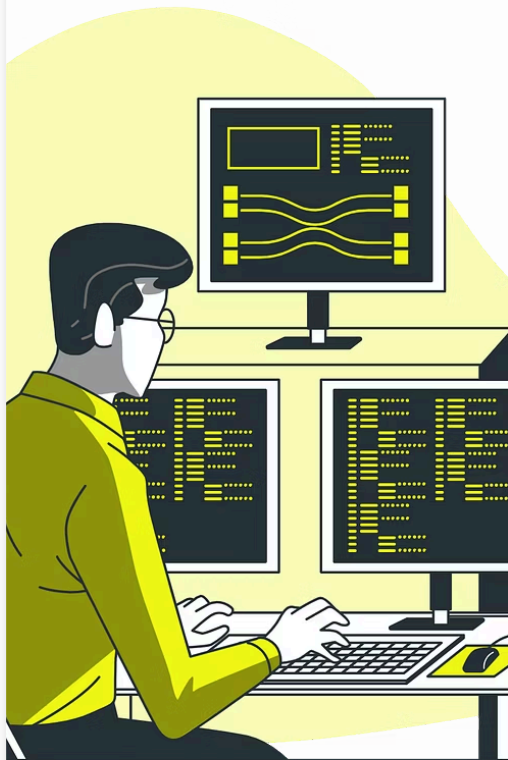
PUT

El método PUT se utiliza para actualizar un recurso existente. Se envía una solicitud al servidor con la URL del recurso a actualizar, junto con los datos actualizados. El servidor reemplaza la representación del recurso con los datos proporcionados. Si el recurso no existe, PUT puede crear uno nuevo, a diferencia de PATCH.

DELETE

El método DELETE se utiliza para eliminar un recurso específico. Se envía una solicitud al servidor con la URL del recurso a eliminar, y el servidor responde con una confirmación de la eliminación o un error si no se puede realizar

la operación.



Endpoints y Recursos

Un endpoint es una URL que representa un recurso o una acción en la API. Facilita la interacción entre cliente y servidor, permitiendo operaciones específicas sobre los recursos. En nuestro caso podría ser:

1 GET /productos

Listar todos los productos

2 GET /productos/{id}

Obtener un producto específico

3 POST /productos

Crear un nuevo producto

Controllers

Creación de Controladores (@RestController)

En Spring Boot, un controlador REST se define con la anotación `@RestController`. Esto indica que los métodos de la clase devolverán el resultado directamente en el cuerpo de la respuesta, usualmente en formato JSON.

Anotaciones Clave

- `@RestController`
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`

Inyección de Dependencias

Usa `@Autowired` en el constructor o atributos para inyectar servicios, repositorios u otras dependencias en el controlador.



Ejemplo de Controlador REST

```
@RestController
@RequestMapping("/productos")
public class ProductoController {
    private final ProductoService productoService;

    @Autowired
    public ProductoController(ProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping
    public List listarProductos() {
        return productoService.listarTodos();
    }
}
```

Endpoints

Endpoints en Java

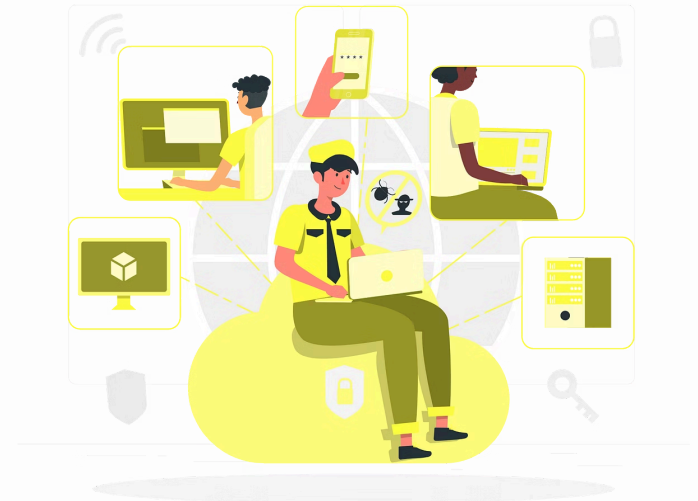
En el ámbito del desarrollo de aplicaciones Java, un endpoint es una dirección URL específica que representa un recurso o una acción dentro de una API. Imagina una API como una interfaz que permite a diferentes aplicaciones comunicarse entre sí.

Un endpoint actúa como una puerta de entrada a esa API, permitiendo que las aplicaciones envíen solicitudes y reciban respuestas.



Ejemplo:

Podríamos tener un endpoint que represente todos los productos de una tienda online, como **/productos**. Cuando una aplicación web envía una solicitud a este endpoint, la API de la tienda procesa la solicitud y envía una respuesta, por ejemplo, una lista de todos los productos disponibles.



Ejemplo de Endpoint con PathVariable

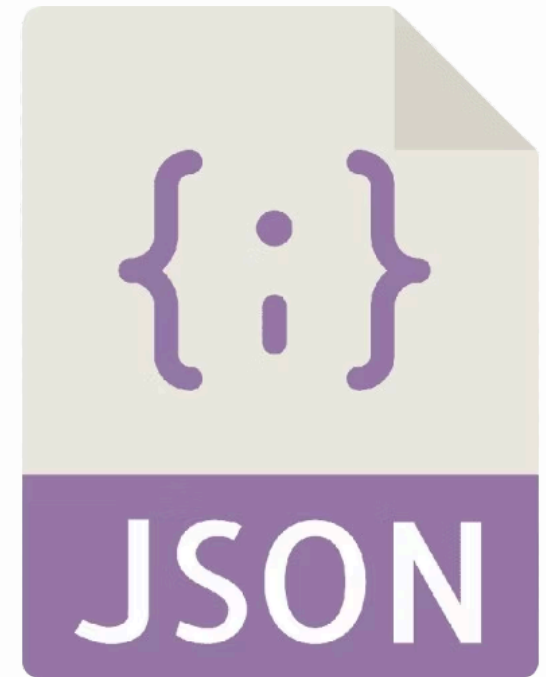
```
@GetMapping("/{id}")
public Producto obtenerProducto(@PathVariable int id) {
    return productoService.obtenerPorId(id);
}
```

Este método captura el ID de la URL y lo utiliza para buscar un producto específico. @PathVariable vincula el parámetro de la ruta al argumento del método.

Json

JSON: El Formato de Intercambio de Datos

JSON, siglas de JavaScript Object Notation, es un formato de texto ligero, legible por humanos, diseñado para intercambiar datos. Es un estándar de facto en la industria, popular en APIs REST debido a su sencillez y versatilidad. Su estructura de datos, basada en objetos y arrays, se adapta fácilmente a la representación de datos complejos, como listas, tablas o jerarquías.



Ejemplo de Manejo de JSON

El siguiente JSON representa un producto de café:

```
{  
  "id": 1,  
  "nombre": "Café de Colombia",  
  "descripcion": "Café 100% Arábica de origen colombiano, con aroma intenso y sabor equilibrado.",  
  "precio": 10.50,  
  "stock": 100  
}
```

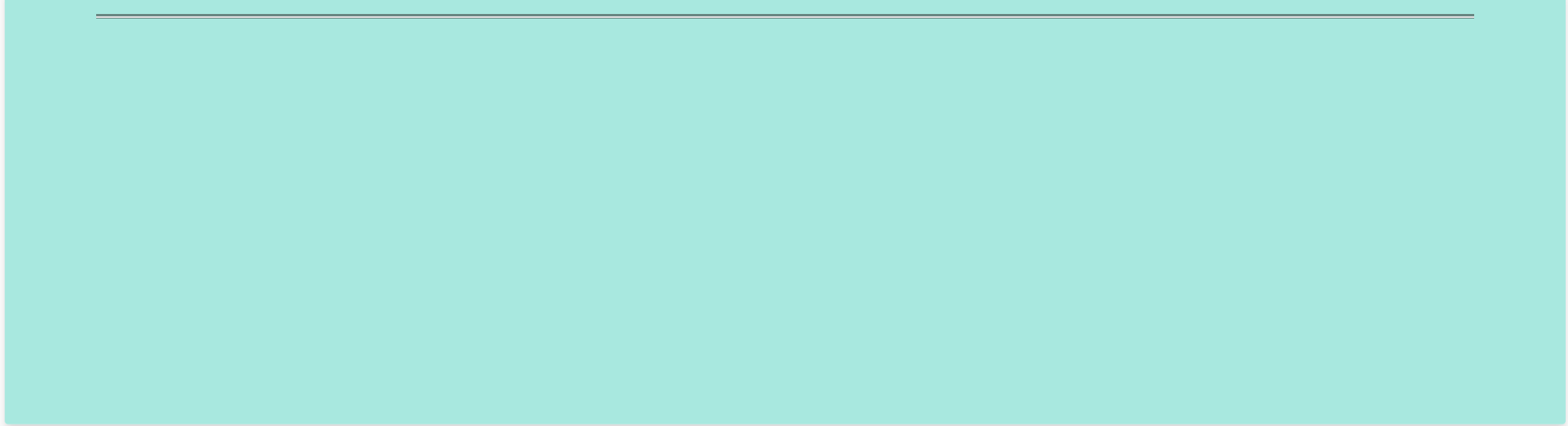
Este JSON contiene información sobre el producto como el ID, nombre, descripción, precio y stock. Este formato es legible por humanos y fácil de procesar por máquinas, lo que lo hace ideal para el intercambio de datos en API REST.

Ejemplo de Manejo de JSON

```
@PostMapping
public Producto crearProducto(@RequestBody Producto nuevoProducto) {
    return productoService.guardar(nuevoProducto);
}
```

@RequestBody indica que el cuerpo de la solicitud HTTP debe ser convertido (deserializado) en un objeto Producto. La respuesta se serializará automáticamente a JSON.

Service



Ejemplo de ProductoService

Un **Service** en Java es una clase que encapsula la lógica de negocio. Actúa como un intermediario entre los controladores y la base de datos, proporcionando una capa de abstracción y reutilización del código.

Por ejemplo, un **ProductoService** podría gestionar las operaciones CRUD (Create, Read, Update, Delete) para productos. A continuación, se muestra un ejemplo simplificado:

Este servicio define métodos para guardar, obtener, actualizar y eliminar productos. Al implementar estos métodos, se puede agregar lógica de negocio, como validaciones o conversiones de datos, sin afectar a los controladores o la base de datos directamente.



Ejemplo de ProductoService

```
@Service
public class ProductoService {
    private List productos = new ArrayList<>();
    private int contadorId = 1;

    public List listarTodos() {
        return productos;
    }

    public Producto guardar(Producto p) {
        p.setId(contadorId++);
        productos.add(p);
        return p;
    }

    // Otros métodos CRUD...
}
```

Preguntas para Reflexionar



Facilidad de Spring Boot

¿Cómo facilita Spring Boot la creación de endpoints REST en comparación con una configuración manual?



Datos en Memoria

¿Que diferencia hay entre trabajar con datos en memoria y datos en una base de datos real?



Ventajas de JSON

¿Qué ventajas aporta el manejo de JSON como formato estándar de intercambio de datos con el frontend?

Ejercicios Prácticos

Situación inicial en TechLab.



Silvia, la Product Owner, quiere que el frontend de la tienda en línea obtenga la lista de productos desde el backend, mostrando así datos reales. Matías y Sabrina preparan un controlador Spring Boot que exponga estos datos, primero desde una lista en memoria, y luego conectarán con una base de datos. Antes de dar el paso a la persistencia, es importante sentar las bases de la API y confirmar que la comunicación funciona correctamente.

Ejercicio Práctico

Optativo | No entregable

Para poder llegar con los tiempos de entrega dispuestos por Silvia, será necesario realizar las siguientes acciones:

Endpoints CRUD en memoria

- Implementá GET /productos, GET /productos/{id}, POST /productos, PUT /productos/{id}, DELETE /productos/{id} en tu controlador.
- Probá estos endpoints con Postman o cURL.

Pruebas con Json

- Envía una solicitud POST el endpoint **/productos** con un cuerpo JSON
- Asegurate que el producto se guarde en la lista y que GET **/productos** lo muestre.